

# 59. Une musique qui reprend au bout

`airFini()` dit quand la partition est arrivée à sa fin.

## 1. Ce qu'il faut comprendre

`airFini(voix)` rend 1 quand la voix a fini sa partition. On relance, et l'on compte les tours — c'est la boucle musicale la plus simple qui soit.

`silence(1)` couperait la voix net, si l'on voulait arrêter au milieu.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <Air>` → [53.1. Air — écrire une musique](#)
- `#include <jouer>` → [53.2. La fonction jouer\(\) — jouer un air tout seul](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <airFini>` → [58.1. La fonction airFini\(\) — l'air est-il fini ?](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1092 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      71 o ← la console, à l'allumage - la mise en route de la console
main()         159 o ← démarrage - ta fonction
VBlank         11 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AvancerAirs    308 o ← VBlank - fait avancer la musique à chaque image
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   49 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         240 o ← CopierTuiles - 15 lettres sur 43
Notes          120 o ← AvancerAirs - la table des notes de musique
"TOURS"        5 o ← texte() - un texte gravé
BOUCLE_0       16 o ← jouer() - une table gravée
```

Lettres gravées : ORSTU0123456789

Pas gravé (jamais appelé) : AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert

Fonctions du langage appelées :

```
jouer()  x 2  104 o sur place, dans main() + BOUCLE_0 (16 o)
nombre() x 2   20 o sur place, dans main() + EcrireNombre (33 o)
texte()  x 1   11 o sur place, dans main() + EcrireTexte (10 o)
airFini() x 1    5 o sur place, dans main()
image()  x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : une musique qui recommence chaque fois qu'elle finit,  
// et compte combien de fois elle a été jouée.  
//  
// airFini(voix) vaut 1 quand l'air de cette voix est arrivé au bout.  
// Dès qu'il est fini, on le relance avec jouer().  
  
#include <Air>          // un air de musique, note par note  
#include <jouer>         // joue un air tout seul  
#include <texte>        // écrit un texte à l'écran  
#include <nombre>       // écrit un nombre en chiffres  
#include <airFini>      // dit si un air est fini
```

## 2. UNE MÉLODIE — BOUCLE

```
Air BOUCLE = {           // 8 pas : do, mi, sol, mi – chacun tenu 2 pas (==).  
    "D04 12", "==", "MI4 12", "==", "SOL4 12", "==", "MI4 12", "==",  
};
```

## 3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

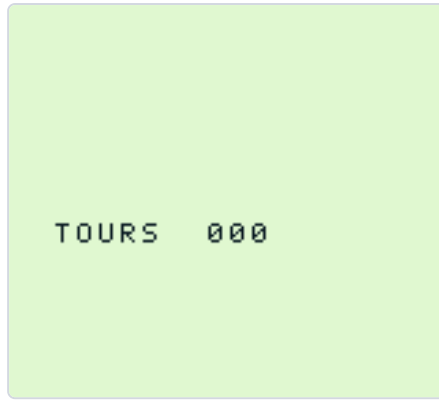
```
int main() {              // Le jeu commence ici.  
    jouer(1, BOUCLE, 10); // Voix 1, un pas toutes les 10 images.  
    uint8_t tours = 0;    // Combien de fois l'air a recommencé.  
  
    // Écrit AVANT la boucle : sans cela, l'écran reste vide une seconde et  
    // demie, et l'on croit que le programme n'a pas démarré.  
    texte(2, 10, "TOURS");  
    nombre(9, 10, tours);  
  
    while (true) {        // Répète pour toujours :  
        if (airFini(1)) { // L'air est fini ?  
            jouer(1, BOUCLE, 10); // on le relance,  
            tours++;              // un tour de plus,  
            nombre(9, 10, tours); // on l'affiche.  
        }  
        image();             // Attend l'image suivante.  
    }  
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180



au bout de sept secondes — image 420

## 4. Ce qu'on doit voir

Le compteur de tours monte à chaque reprise.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la partition est arrivée au bout (1 tour(s))
- ✓ et elle a repris (2)
- ✓ le compteur est affiché

## 6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

BOUCLE

+ air

MIDI

WAV

BOUCLE

Écouter

note

– silence

= tenir

▼

▲

– pas

+ pas

8

images par pas

D05

D04

vol

0

4

`jouer(1, BOUCLE, 8);` — le quatrième argument, 1, le fait recommencer sans fin.

Clique la partition pour poser une note ; le bas de la grille règle le **volume** du pas.

Le clic droit fait taire un pas. `==` tient la note d'avant — c'est ainsi qu'on écrit une blanche.

Ensuite : **59.1. Chapitre 9 — Ranger ce qu'on manipule : ce qui vient** — chapitre 9 : Ranger ce qu'on manipule. · [gameboy3](#) — écrire une cartouche Game Boy en C++